EXPERIMENT NO : 1

AIM: Case study of any existing successful AI system

Theory: Gemini

Working and History (Development of the Application)

Background and Motivation

With the rapid growth of data across domains, analyzing large datasets has become increasingly complex. Traditional data analysis and visualization tools often require technical expertise, making them inaccessible to non-technical users. To address this challenge, the paper introduces Visistant, a system designed to democratize data visualization by leveraging Large Language Models (LLMs). The goal is to enable users to generate meaningful visualizations and insights using natural language, without requiring programming knowledge

Working Principle

Core Engine

Visistant is powered by Google's Gemini Pro 1.0, which generates Python code using the Plotly library based on natural language queries. For extracting insights from visual outputs, Gemini Pro Vision is used to analyze images of the generated charts

System Architecture

- Streamlit is used to build the interactive web-based user interface
- LangChain manages conversational context and memory, allowing users to interact with the system in a continuous and natural manner

Process Flow

1. **Data Input:** Users upload one or more tabular datasets (CSV format) through the Streamlit interface
2. **Prompt Construction:** The system generates an initial prompt containing dataset metadata such as column names, data types, and unique values

3. **Query Processing:** The user's natural language query is combined with the dataset context and sent to Gemini Pro.

4. **Visualization Generation:** Gemini Pro generates Python code using Plotly, which is executed to produce interactive visualizations directly within the application.

5. **Insight Extraction:** The generated visualization is converted into an image and passed to Gemini Pro Vision to identify patterns, trends, and outliers, producing automated insights.

Features

1. **Conversational Interface:**
   Users interact with Visistant in a chatbot-like environment, allowing follow-up questions and iterative refinement of visualizations

2. **Interactive Visualizations:**
   Unlike traditional systems that generate static plots, Visistant produces interactive charts with features such as zooming, panning, and hover-based tooltips using Plotly

3. **Automated Data Handling:**
   The system supports multiple CSV uploads and performs basic data cleaning, including duplicate removal and handling missing values

4. **Context Awareness:**
   By employing windowed conversational memory through LangChain, Visistant maintains context across interactions, enabling a seamless analytical workflow

5. **Automated Insight Generation:**
   A toggle option allows users to generate textual insights derived from visualizations using Gemini Vision

Advantages

- **Easy to Use:** You don't need to know coding or complex tools; you can just ask in normal language
- **Interactive Charts:** Unlike old systems, it uses Plotly, so you can zoom, pan, and hover over the charts to see details

- **Handles Mistakes:** The system is smart enough to understand spelling errors or vague questions (like understanding "tomatoes" means "Rotten Tomatoes Rating")
- **Cost-Effective:** It is designed to use fewer "tokens," making it cheaper to run compared to similar tools like Chat2VIS

Disadvantages

- **Closed Source Model:** While the app interface is open, the main engine (Gemini) is not open-source, so it cannot be fully modified
- **Inconsistent Results:** Sometimes, the AI might give slightly different charts for the exact same question if you run it again

Limitations

- **Loves Bar Charts:** The model is biased towards bar charts and uses them even when a pie chart might be better
- **Variability:** Even with the same settings, the AI sometimes changes the plot type on its own during re-runs
- **Spelling Sensitive in Advanced Mode:** In advanced mode if you make a small spelling mistake in a column name, the system will show an error

Future Scope

- **Better Training Data:** Future work involves creating a proper dataset specifically for training AI on Plotly code
- **Human Testing:** The system needs to be tested by real people and experts to see how good it is in real-world use
- **New Benchmarks:** Researchers plan to create better testing standards (benchmarks) that cover more complex, real-life scenarios

References

[1] G. K. S. Ram and V. Muthumanikandan, "Visistant: A Conversational Chatbot for Natural Language to Visualizations With Gemini Large Language Models," *IEEE Access*, 2024.

[2] P. Maddigan and T. Susnjak, "Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models," *IEEE Access*, 2023.

[3] C. Wang, J. Thompson, and B. Lee, "Data Formulator: AI-Powered Concept-Driven Visualization Authoring," *IEEE Transactions on Visualization and Computer Graphics*, 2024.

Conclusion:In conclusion, this study successfully introduced Visistant, a chatbot system that uses the Gemini Pro model to turn natural language directly into visualizations. The system proved to be very effective because it can remember previous conversations, which helps it understand the context better and give accurate results. When compared to other tools like LIDA and Chat2VIS, Visistant performed better by creating more interactive plots using the Plotly library and was also found to be more economical. Overall, this research helps make data analysis easier for everyone, even those without technical skills. Future work will focus on creating a larger training dataset and testing the system with real users to improve it further

Experiment No.2

AIM:

To provide the PEAS description and TASK Environment for the AI application — Gemini.

THEORY:

1. What is a PEAS Descriptor?

PEAS is an AI agent representation system that focuses on evaluating the **Performance measure, Environment, Actuators, and Sensors** of an intelligent agent.

PEAS helps in clearly defining the **task environment** of an AI system so that appropriate algorithms can be designed.

PEAS stands for:

- **P – Performance Measure**
- **E – Environment**
- **A – Actuators**
- **S – Sensors**

An intelligent agent:

- Operates autonomously
- Interacts with the environment
- Takes actions using actuators
- Perceives surroundings using sensors
- Aims to achieve predefined goals

2. PEAS Description for the Application Chosen

Application: Google Gemini — AI Conversational & Multimodal Assistant

P – Performance Measure

The performance of the Gemini AI system is evaluated based on:

- Accuracy and correctness of responses
- Relevance to user queries
- Context understanding and coherence
- Ability to handle multimodal inputs (text, images, files)
- Response clarity and completeness
- User satisfaction
- Ethical compliance and safe outputs
- Speed and response efficiency

E – Environment

The environment in which Gemini operates includes:

- Digital platforms (web applications, mobile apps, APIs)
- Cloud computing infrastructure
- User-generated prompts and files
- Conversation history
- Various domains such as education, coding, business, healthcare, research, and creativity
- Dynamic real-time interaction environment

The environment is virtual and continuously evolving based on user interaction.

A – Actuators

Actuators are the actions performed by Gemini in its environment:

- Generating text responses
- Answering questions
- Explaining concepts
- Writing and debugging code
- Translating languages
- Summarizing documents
- Analyzing images (multimodal capability)

- Providing recommendations
- Engaging in conversation

These actions influence the user's decisions and knowledge.

S – Sensors

Sensors provide input data to Gemini:

- User text prompts
- Uploaded images
- Documents and files
- Conversation history
- API instructions and system settings

These inputs help the system interpret and respond effectively.

3. What is the TASK Environment?

A task environment describes:

- The world in which the agent operates
- The tasks it must perform
- The conditions under which it functions

Task environments are classified using the following properties:

1. Fully Observable vs. Partially Observable
2. Single Agent vs. Multiagent
3. Deterministic vs. Stochastic
4. Episodic vs. Sequential
5. Static vs. Dynamic
6. Discrete vs. Continuous

4. TASK Environment for Google Gemini

The task environment of Gemini can be classified as follows:

1. Fully Observable vs. Partially Observable

Type: Partially Observable

Description:
Gemini does not have complete access to the real-world environment or user intentions.

Example:

- It only knows what the user provides.
- Missing context can affect output.
- User goals may not always be clearly stated.

2. Single Agent vs. Multiagent

Type: Single Agent

Description:
Gemini functions as one intelligent AI agent interacting with users.

Example:

- It independently processes user inputs and generates outputs.
- It does not compete or cooperate with other AI agents in standard operation.

3. Deterministic vs. Stochastic

Type: Stochastic

Description:

The system produces outputs probabilistically.

Example:

- The same question may produce slightly different answers.
- Creative tasks generate varied responses.

4. Episodic vs. Sequential

Type: Sequential

Description:
Current responses depend on previous conversation history.

Example:

- Follow-up questions rely on earlier context.
- Dialogue evolves over time.

5. Static vs. Dynamic

Type: Dynamic

Description:
The environment changes as new user inputs are provided.

Example:

- Each prompt updates the conversation state.
- Context continuously evolves.

6. Discrete vs. Continuous

Type: Discrete

Description:
Gemini processes discrete tokens (words or subwords).

Example:

- Text is divided into countable tokens.
- Output generation is based on token prediction.

CONCLUSION: In this experiment, the PEAS descriptor and Task Environment for the Gemini-based visualization assistant were successfully analyzed. The system functions as a **single-agent, sequential, and partially observable environment**. Understanding these properties is important, as they influence system design choices such as prompt refinement and memory handling, ensuring effective and accurate user interactions.

Name : Jignesh Bhatia Class : TE A1 Comps Roll no. :11

Experiment No. 3

Aim : Study and Installation of Prolog

Theory:

What is Logic Programming?

Logic Programming is a programming paradigm based on **formal logic**.
In logic programming, a program is written using:

- **Facts** (basic truths)
- **Rules** (logical relationships)
- **Queries** (questions to get answers)

The system uses logical reasoning to find answers.

The most common logic programming language is **Prolog**.

Example: Family Relationship System

Let us understand logic programming using a simple family example.

1. Facts

Facts are statements that are always true.
They represent information about objects and relationships.

General form:

predicate(object1, object2).

Example Facts:

parent(ram, sita).
parent(ram,
mohan). parent(sita,
rita). female(sita).
female(rita).
male(ram).
male(mohan).

Meaning:

- Ram is parent of Sita and Mohan.
- Sita is parent of Rita.
- Sita and Rita are females.
- Ram and Mohan are males.

Each fact ends with a **period (.)**

2. Rules

Rules define relationships using facts.
They tell that something is true **if some conditions are true**.

General form:

conclusion :- condition.

It is read as:
"Conclusion is true IF condition is true."

Example Rules

Rule 1: Mother

mother(X, Y) :- parent(X, Y), female(X).

Meaning:
X is mother of Y if

- X is parent of Y
- AND X is female

Rule 2: Father

father(X, Y) :- parent(X, Y), male(X).

Rule 3: Grandparent

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

Meaning:
X is grandparent of Z if

- X is parent of Y
- AND Y is parent of Z

3. Queries

Queries are questions asked to the system.
They start with ?-

Query 1:

?- mother(sita, rita).

Answer:

Yes

Query 2:

?- father(ram, sita).

Answer:

Yes

Query 3:

?- grandparent(ram, rita).

Answer:

Yes

(Because Ram → Sita → Rita)

Query with Variable:

?- parent(ram, X).

Output:

X = sita ;

X = mohan.

This means Ram has two children: Sita and Mohan.

How Logic Programming Works

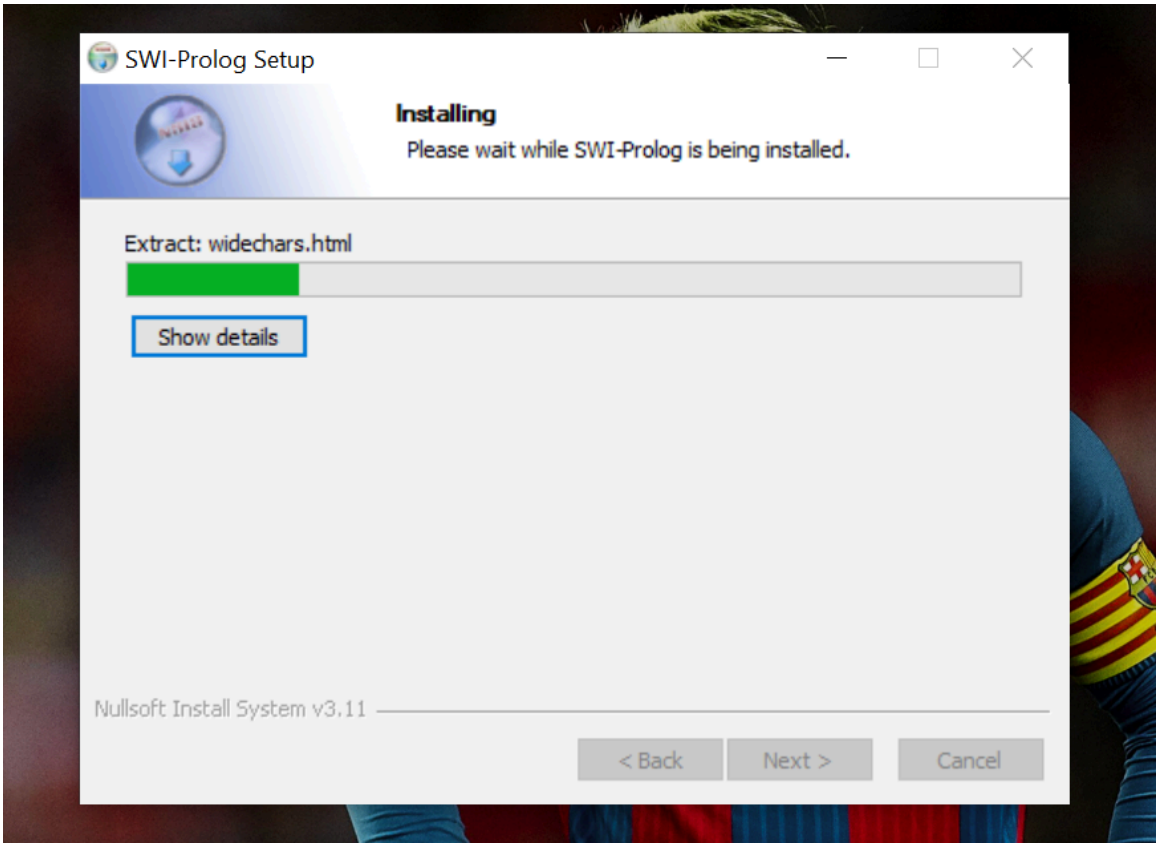
Logic programming uses:

1. **Unification** – Matching queries with facts and rules
2. **Backtracking** – Searching for all possible solutions
3. **Inference Engine** – Deriving new facts from existing facts and rules

Advantages of Logic Programming

- Easy to represent relationships
- Less code compared to procedural programming
- Good for AI and expert systems
- Declarative (focus on *what* to solve, not *how*)

Installation of Prolog:



Code1:

ram likes mango.
seema is a girl.
bill likes cindy.
rose is red.
john owns gold.

likes(ram,
mango).
girl(seema).
red(rose).
likes(bill, cindy).
owns(john, gold).

OUTPUT:

```
true.

21 ?- likes(ram, X).
X = mango.

22 ?- girl(seema).
true.

23 ?- girl(S).
S = seema.

24 ?- red(Y)
|    .
Y = rose.

25 ?- owns(X,Y).
X = john,
Y = gold.

26 ?- likes(bill,C).
C = cindy.
```

CODE2:

```
rainy(seattle).

rainy(rochester).

cold(rochester).

snowy(X) :- rainy(X), cold(X).
```

OUTPUT:

```
27 ?- consult(['c:/Users/STUDENT/Downloads/exp3.pl']).
true.

28 ?- rainy(C)
|    .
C = seattle .

29 ?- snowy(C).
C = rochester.
```

Conclusion

Logic Programming is a programming approach based on logic and reasoning.
A program consists of:

- **Facts** (basic truths)
- **Rules** (logical conditions)
- **Queries** (questions to get results)

Using facts and rules, the system automatically derives answers to queries through logical inference.

Thus, Logic Programming is widely used in Artificial Intelligence, expert systems, and knowledge-based systems.

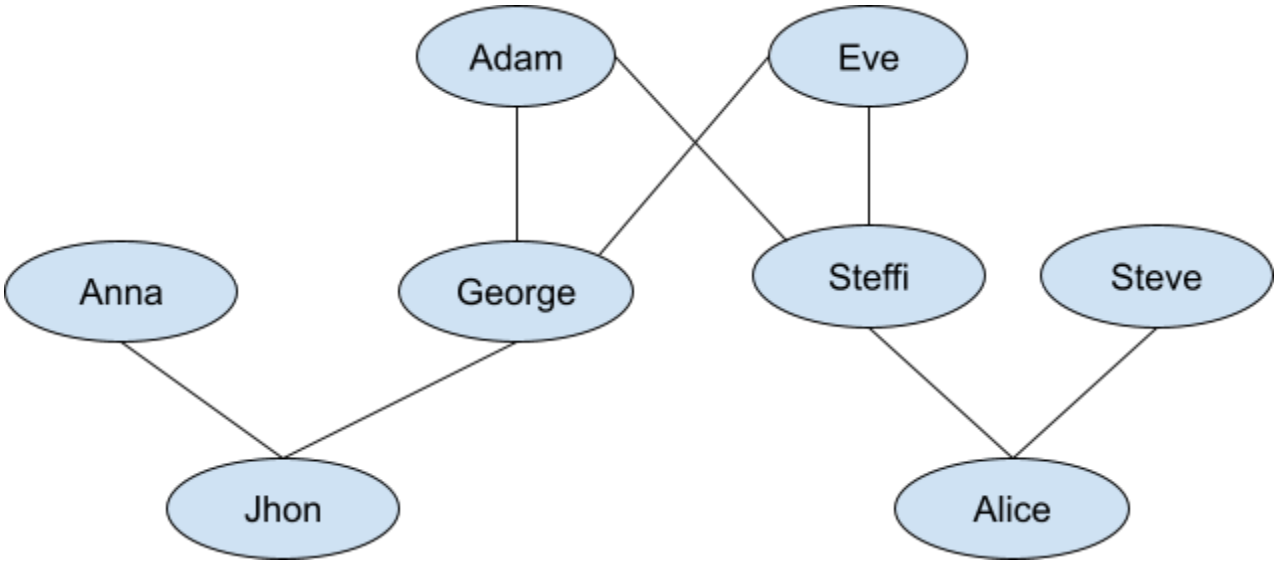
Experiment No. 04

Aim : To implement Family Tree in Prolog

Theory :

A **Family Tree** represents relationships among family members using facts and rules.

We create a family tree starting from **Adam and Eve**, who are considered the first parents in this tree.



First Generation:

- Adam (Male)
- Eve (Female)

Second Generation:

Children of Adam and Eve:

- George (Male)
- Steffi (Female)

Third Generation:

- George + Anna → John
- Steffi + Steve → Alice

Explanation of Prolog Code

Facts (Basic Information)

Male Members

male(adam).

male(george).

male(steve).

male(john).

These facts define who is male.

Female Members

female(alice).

female(anna).

female(eve).

female(steffi).

These facts define who is female.

Parent Relationships

parent(adam,george).

parent(eve,george).

parent(adam,steffi).

parent(eve,steffi).

parent(george,john).

parent(anna,john).

parent(steffi,alice).

parent(steve,alice).

This defines who is the parent of whom.

Example:

parent(adam,george).

This means Adam is the parent of George.

Rules (Derived Relationships)

Mother Rule

mother(X,Y):- parent(X,Y), female(X).

Meaning:
X is mother of Y if:

- X is parent of Y
- X is female

Example Query:

?- mother(eve,george).

Output:

true.

Father Rule

father(X,Y):- parent(X,Y), male(X).

X is father of Y if:

- X is parent of Y
- X is male

Sister Rule

sister(X,Y):- parent(Z,X), parent(Z,Y), female(X), X\==Y.

X is sister of Y if:

- They share the same parent
- X is female
- X and Y are not same person

Brother Rule

brother(X,Y):- parent(Z,X), parent(Z,Y), male(X), X\==Y.

Same logic as sister but X must be male.

Grandfather Rule

grandfather(X,Z):- father(X,Y), parent(Y,Z).

X is grandfather of Z if:

- X is father of Y
- Y is parent of Z

Example:

Adam → George → John

So Adam is grandfather of John.

Grandmother Rule

grandmother(X,Z):- mother(X,Y), parent(Y,Z).

Same logic for grandmother.

Siblings Rule

siblings(X,Y):- (brother(X,Y);sister(X,Y)), X\==Y.

X and Y are siblings if:

- X is brother or sister of Y
- They are not the same person

Uncle Rule

uncle(X,Y):- parent(Z,Y), brother(X,Z).

X is uncle of Y if:

- Z is parent of Y
- X is brother of Z

Example:
George is uncle of Alice.

Aunty Rule

aunty(X,Y):- parent(Z,Y), sister(X,Z).

X is aunty of Y if:

- Z is parent of Y
- X is sister of Z

Cousin Rule

cousin(X,Y):- parent(A,X), parent(B,Y), siblings(A,B).

X and Y are cousins if:

- Their parents are siblings.

Example:
John and Alice are cousins.

Code for Family tree of Adam and Eve:

```
male(adam).
male(george).
male(steve).
male(john).
```

```
female(alice).
female(anna).
female(eve).
female(steffi).

parent(adam,george).
parent(eve,george).
parent(adam,steffi).
parent(eve,steffi).
parent(george,john).
parent(anna,john).
parent(steffi,alice).
parent(steve,alice).

mother(X,Y):- parent(X,Y), female(X).

father(X,Y):- parent(X,Y), male(X).

sister(X,Y):- parent(Z,X), parent(Z,Y), female(X),X\==Y.

brother(X,Y):- parent(Z,X), parent(Z,Y), male(X),X\==Y.

grandfather(X,Z):- father(X,Y), parent(Y,Z).

grandmother(X,Z):- mother(X,Y), parent(Y,Z).

siblings(X,Y):- (brother(X,Y);sister(X,Y)),X\==Y.

uncle(X,Y):- parent(Z,Y),brother(X,Z).

aunty(X,Y):- parent(Z,Y),sister(X,Z).

cousin(X,Y):- parent(A,X),parent(B,Y),siblings(A,B).
```

Prolog code for my family tree:

```
male(danaji).
male(shankarlal).
male(chhogalalji).
male(jignesh).
male(laxman).

female(shantidevi).
female(geeta).
female(badidevi).

parent(danaji,shankarlal).
parent(shantidevi,shankarlal).

parent(danaji,geeta).
parent(shantidevi,geeta).

parent(shankarlal,jignesh).
parent(badidevi,jignesh).

parent(geeta,laxman).
parent(chhogalalji,laxman).

mother(X,Y):- parent(X,Y), female(X).
father(X,Y):- parent(X,Y), male(X).

sister(X,Y):- parent(Z,X), parent(Z,Y), female(X), X \== Y.
brother(X,Y):- parent(Z,X), parent(Z,Y), male(X), X \== Y.

grandfather(X,Z):- father(X,Y), parent(Y,Z).
grandmother(X,Z):- mother(X,Y), parent(Y,Z).

siblings(X,Y):- (brother(X,Y);sister(X,Y)), X \== Y.

uncle(X,Y):- parent(Z,Y), brother(X,Z).
aunty(X,Y):- parent(Z,Y), sister(X,Z).

cousin(X,Y):- parent(A,X), parent(B,Y), siblings(A,B).
```

Output :

```
SWI-Prolog console
File  Settings  Tools  Debug  Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.2-27-gc527bb29a-DIRTY)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

101 ?- XopenImage(@ms_left_arrow_image/image)
101 ?- consult(['c:/Users/STUDENT/Desktop/familyyyyy.pl']).
true.

102 ?- father(X,jignesh).
X = shankarlal .

103 ?- mother(X,jignesh).
X = badidevi.

104 ?- brother(X,geeta).
X = shankarlal .

105 ?- sister(X,shankarlal).
X = geeta .

106 ?- grandfather(X,jignesh).
X = danaji .

107 ?- grandmother(X,jignesh).
X = shantidevi
```

Conclusion: Thus we have successfully implemented family tree in prolog.

Experiment No: 5

AIM: Implement Uninformed search techniques (DFS, BFS).

THEORY:

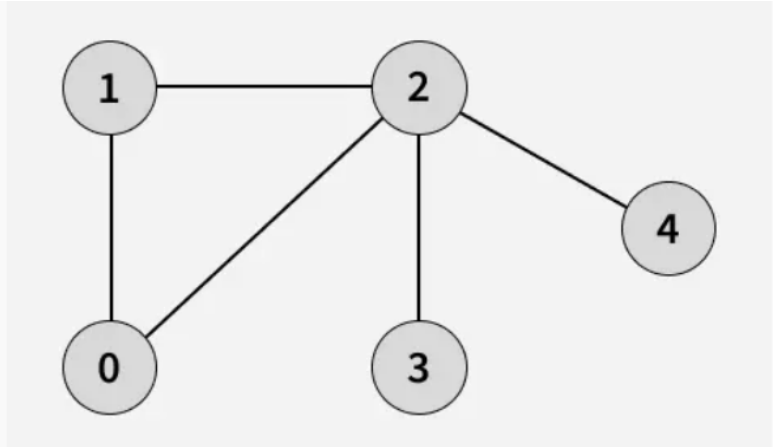
What is BFS? Explain with example.

Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

- BFS is different from DFS in a way that closest vertices are visited before others. We mainly traverse vertices level by level.
- Popular graph algorithms like Dijkstra's shortest path, Kahn's Algorithm, and Prim's algorithm are based on BFS.
- BFS itself can be used to detect cycle in a directed and undirected graph, find shortest path in an unweighted graph and many more problems.

Example:

Input: adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]



Output: [0, 1, 2, 3, 4]
Explanation: Starting from 0, the BFS traversal proceeds as follows:
Visit 0 - Print: 0
Visit 1 (neighbor of 0) - Print: 1
Visit 2 (next neighbor of 0) - Print: 2
Visit 3 (first neighbor of 2 that hasn't been visited yet) - Print: 3
Visit 4 (next neighbor of 2) - Print: 4

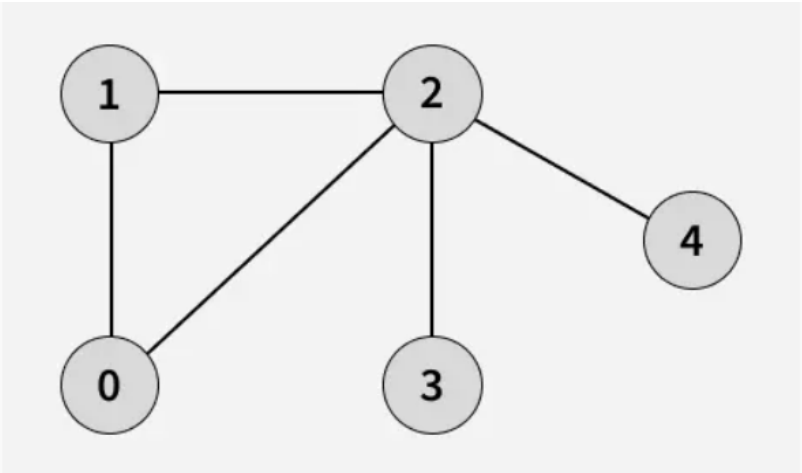
What is DFS? Explain with example.

Depth First Search (DFS) is a graph traversal method that starts from a source vertex and explores each path completely before backtracking and exploring other paths. To avoid revisiting nodes in graphs with cycles, a visited array is used to track visited vertices.

Note: There can be multiple DFS traversals of a graph according to the order in which we pick adjacent vertices. Here we pick vertices as per the insertion order.

Example:

Input: adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]



Output: [0, 1, 2, 3, 4]
Explanation: The source vertex is 0. We visit it first, then we visit its adjacent.
Start at 0: Mark as visited. Print 0.
Move to 1: Mark as visited. Print 1.
Move to 2: Mark as visited. Print 2.
Move to 3: Mark as visited. Print 3.(backtrack to 2)
Move to 4: Mark as visited. Print 4(backtrack to 2, then backtrack to 1, then to 0).

Note that there can be more than one DFS Traversals of a Graph. For example, after 1, we may pick adjacent 2 instead of 0 and get a different DFS.

BFS Code:

```
from collections import deque


def bfs(graph, start):
    visited = set()
    queue = deque([start])
```

```
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# User Input
graph = {}

v = int(input("Enter number of vertices: "))
for i in range(v):
    node = input("Enter vertex name: ")
    graph[node] = []

e = int(input("Enter number of edges: "))
for i in range(e):
    u, v = input("Enter edge (u v): ").split()
    graph[u].append(v)
    graph[v].append(u)

start = input("Enter starting vertex: ")

print("BFS Traversal:")
bfs(graph, start)
```

OUTPUT:

```
Enter number of vertices: 6
Enter vertex name: T
Enter vertex name: K
Enter vertex name: A
Enter vertex name: B
Enter vertex name: C
Enter vertex name: D
Enter number of edges: 9
Enter edge (u v): T K
Enter edge (u v): K C
Enter edge (u v): D A
Enter edge (u v): B D
Enter edge (u v): A B
Enter edge (u v): K T
Enter edge (u v): C A
Enter edge (u v): B A
Enter edge (u v): D C
Enter starting vertex: T
BFS Traversal:
T K C A D B
=== Code Execution Successful ===
```

DFS Code:

```
def dfs(graph, node, visited):
    visited.add(node)
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

# User Input
graph = {}

v = int(input("Enter number of vertices: "))
for i in range(v):
    node = input("Enter vertex name: ")
    graph[node] = []
```

```
e = int(input("Enter number of edges: "))
for i in range(e):
    u, v = input("Enter edge (u v): ").split()
    graph[u].append(v)
    graph[v].append(u)

start = input("Enter starting vertex: ")
visited = set()

print("DFS Traversal:")
dfs(graph, start, visited)
```

OUTPUT:

```
Enter number of vertices: 4
Enter vertex name: K
Enter vertex name: T
Enter vertex name: A
Enter vertex name: B
Enter number of edges: 4
Enter edge (u v): K T
Enter edge (u v): A B
Enter edge (u v): B T
Enter edge (u v): A K
Enter starting vertex: K
DFS Traversal:
K T B A
=== Code Execution Successful ===
```

CONCLUSION:

The experiment successfully implemented DFS and BFS algorithms and demonstrated how uninformed search techniques are used to traverse and search nodes in a graph.

Experiment No: 6

AIM: Implement Informed search Technique (A*).

THEORY:

What is A* Algorithm?

The A* (A-Star) Algorithm is a search algorithm used to find the shortest path between a starting node and a goal node. It is widely used in Artificial Intelligence, pathfinding, and graph traversal (for example in maps, games, and robotics).

A* combines the advantages of Uniform Cost Search and Greedy Best-First Search. It selects the node with the lowest estimated total cost using the formula:

$$f(n) = g(n) + h(n)$$

Where:

- g(n) = Actual cost from the start node to node n
- h(n) = Heuristic estimate of cost from node n to goal
- f(n) = Total estimated cost of the cheapest solution through node n

Algorithm:

The A* search algorithm proceeds in the following steps :

Step 1: Initialize: Initialize the open list with the start node and the closed list as empty. The open list contains nodes to be evaluated, while the closed list contains nodes that have already been evaluated.

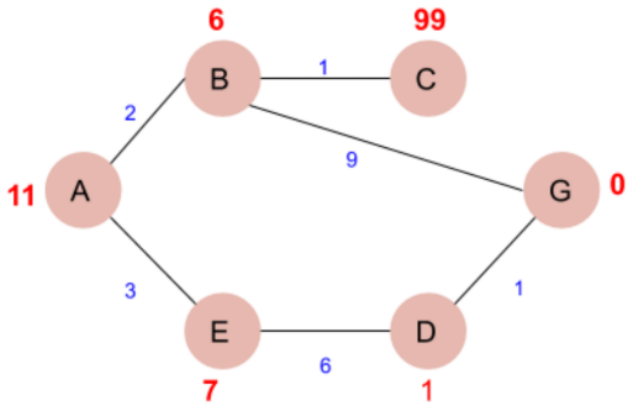
Step 2: Select Node: Select the node with the lowest f value (f = g + h) from the open list and move it to the closed list.

Step 3: Check Goal: If the selected node is the goal node, reconstruct the path and return it as the solution.

Step 4: Expand Node: For each neighbor of the selected node, calculate its g, h, and f values. If the neighbor is not in the open list, add it. If it is already in the open list with a higher f value, update its f value and parent node.

Step 5: Repeat: Repeat steps 2 to 4 until the open list is empty or the goal node is found.

Example:



Step 1: Start at Node A

- Path: A
- g(A): 0
- h(A): 11
- f(A): 0 + 11 = 11
- Neighbors: B, E

Step 2: Evaluate Neighbors of A

- **Path A → B:**
  - g(B) = 2
  - h(B) = 6
  - f(B) = 2 + 6 = **8**
- **Path A → E:**
  - g(E) = 3
  - h(E) = 7
  - f(E) = 3 + 7 = **10**
- **Decision:** Since f(B) < f(E), we explore Node B first.

Step 3: Evaluate Neighbors of B

From B, we can go to C or G.

- **Path A → B → C:**
  - g(C) = 2 (A to B) + 1 (B to C) = 3
  - h(C) = 99
  - f(C) = 3 + 99 = 102
- **Path A → B → G:**
  - g(G) = 2 (A to B) + 9 (B to G) = 11
  - h(G) = 0
  - f(G) = 11 + 0 = 11
- **Decision:** Compare unexplored nodes: f(E)=10, f(G)=11, f(C)=102. Since f(E) is the smallest, the algorithm selects E for expansion.

Step 4: Evaluate Neighbors of E

From E, we move to D.

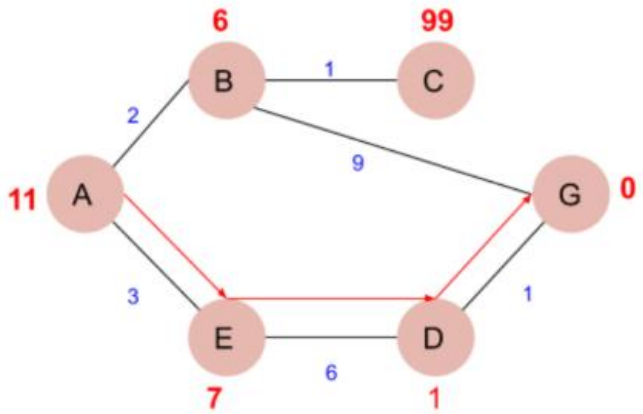
- **Path A → E → D:**
  - g(D) = 3 (A to E) + 6 (E to D) = 9
  - h(D) = 1
  - f(D) = 9 + 1 = 10

Step 5: Evaluate Neighbors of D

From D, we move to G.

- **Path A → E → D → G:**
  - g(G) = 9 (A to D) + 1 (D to G) = 10
  - h(G) = 0
  - f(G) = 10 + 0 = 10

Optimal Path: A → E → D → G
Total Cost: 10



CODE:

```
def aStarAlgo(start_node, stop_node):

 open_set = set([start_node])

 closed_set = set()

 g = {}

 parents = {}

 g[start_node] = 0

 parents[start_node] = start_node
```

```
while len(open_set) > 0:
 n = None

 for v in open_set:
  if n is None or g[v] + heuristic(v) < g[n] + heuristic(n):
   n = v

 if n is None:
  print("Path does not exist!")
  return None

 if n == stop_node:
  path = []
  while parents[n] != n:
   path.append(n)
   n = parents[n]
  path.append(start_node)
  path.reverse()
  print("Path found:", path)
  return path

 for (m, weight) in get_neighbors(n):
  if m not in open_set and m not in closed_set:
   open_set.add(m)
```

```
    parents[m] = n
    g[m] = g[n] + weight
   else:
    if g[m] > g[n] + weight:
     g[m] = g[n] + weight
     parents[m] = n
     if m in closed_set:
      closed_set.remove(m)
      open_set.add(m)

  open_set.remove(n)
  closed_set.add(n)

 print("Path does not exist!")
 return None

def get_neighbors(v):
 if v in Graph_nodes:
  return Graph_nodes[v]
 else:
  return []

def heuristic(n):
 H_dist = {'A':11,'B':6,'C':99,'D':1,'E':7,'G':0}
 return H_dist[n]
```

```
Graph_nodes={
 'A':[('B',2),('E',3)],
 'B':[('A',2),('C',1),('G',9)],
 'C':[('B',1)],
 'D':[('E',6),('G',1)],
 'E':[('A',3),('D',6)],
 'G':[('B',9),('D',1)]
}

aStarAlgo('A','G')
```

OUTPUT:

```
Output                                    Clear
Path found: ['A', 'E', 'D', 'G']

=== Code Execution Successful ===
```

CONCLUSION:

The A* informed search algorithm was successfully implemented to find the optimal path between the start node and the goal node. By using the evaluation function $f(n) = g(n) + h(n)$, the algorithm efficiently selected the best node to explore, resulting in the shortest path from A to G with minimum cost.

EXPERIMENT NO. 07

AIM: Implementation of MIN-MAX Algorithm

THEORY:

The Minimax algorithm is a decision-making algorithm commonly used in two-player, turn-based games such as tic-tac-toe, chess, and checkers. It operates under the assumption that both players play optimally.

The algorithm models the game as a tree structure, where:

- Each node represents a game state.
- The root node is the current state.
- Leaf nodes represent terminal states (win, loss, or draw).

There are two types of players:

- Maximizing player (MAX): aims to maximize the score.
- Minimizing player (MIN): aims to minimize the score.

The algorithm works recursively:

1. It explores all possible moves from the current state.
2. At each level:
    - MAX chooses the move with the maximum value.
    - MIN chooses the move with the minimum value.
3. The process continues until it reaches terminal states.
4. Utility values are assigned to terminal nodes:
    - Win → positive value
    - Loss → negative value
    - Draw → zero

5. These values are then propagated upward to determine the best move.

CODE:

```
import math
def minimax(depth, nodeIndex, isMax, values, maxDepth):
    # Base case
    if depth == maxDepth:
        return values[nodeIndex]
    if isMax:
        left = minimax(depth + 1, nodeIndex * 2, False, values, maxDepth)
        right = minimax(depth + 1, nodeIndex * 2 + 1, False, values, maxDepth)
        return max(left, right)
    else:
        left = minimax(depth + 1, nodeIndex * 2, True, values, maxDepth)
        right = minimax(depth + 1, nodeIndex * 2 + 1, True, values, maxDepth)
        return min(left, right)
# -------- USER INPUT --------
n = int(input("Enter number of leaf nodes (must be power of 2): "))
values = []
print("Enter values of leaf nodes:")
for i in range(n):
    val = int(input(f"Value {i+1}: "))
    values.append(val)
# calculate depth
depth = int(math.log2(n))
# run minimax
result = minimax(0, 0, True, values, depth)
print("\nOptimal value (Minimax):", result)
```

OUTPUT:

```
PS C:\Users\STUDENT\Desktop\aiexp7> & C:\Users\STUDENT\AppData\Local\Microsoft\Windows
Apps\python3.11.exe c:/Users/STUDENT/Desktop/aiexp7/exp7.py
Enter number of leaf nodes (must be power of 2): 8
Enter values of leaf nodes:
Value 1: 3
Value 2: 5
Value 3: 2
Value 4: 9
Value 5: 12
Value 6: 5
Value 7: 23
Value 8: 23

Optimal value (Minimax): 12
PS C:\Users\STUDENT\Desktop\aiexp7>
```

CONCLUSION :

The experiment demonstrates how the Minimax algorithm systematically evaluates all possible game states to determine the optimal move in a competitive environment. By recursively applying maximizing and minimizing strategies, the algorithm anticipates the opponent's actions and selects the best possible decision.

EXP NO. 08

AIM : Programs using Prolog for Mathematic operations, using function and Input/output function as an AI Programming language.

CODE :

1. **Prolog program using Mathematic AI operations**

OUTPUT :

```
?- X is 5+2.
X = 7.

?- X is 5-8.
X = -3.

?- X is 28*13.
X = 364.

?- X is 28/13.
X = 2.1538461538461537.

?- X is 5 mod 10.
X = 5.

?- X is 532.
X = 532.

?- X is 5^8.
X = 390625.

?- X is 5^2.
X = 25.

?- 26 is 5^2.
false.

?- X is 5*3, Y is X*20.
X = 15,
```

```
Y = 300.
```

2. Prolog program using functions.

CODE:

```
person(jignesh,5.1,black,white).
person(Rohit, 5.0, black,white).
person(system, 4.9, white,brown).
person(Allen, 5.1, white,white).
```

OUTPUT:

```
?- person(X,Y,Z,W).
X = jignesh,
Y = 5.1,
Z = black,
W = white ;
X = Rohit,
Y = 5.0,
Z = black,
W = white ;
X = system,
Y = 4.9,
Z = white,
W = brown ;
X = Allen,
Y = 5.1,
Z = W, W = white
```

3.

CODE:

```
food(burger).
food(sandwich).
food(pizza).
lunch(sandwich).
dinner(pizza).
meal(X):- food(X).
```

OUTPUT:

```
?- food(What).
```

```
What = burger ;
What = sandwich ;
What = pizza.

?- food(burger).
true.

?- dinner(pizza).
true.

?- meal(X),lunch(X).
X = sandwich .

?- meal(X),dinner(X).
X = pizza.

?- lunch(pizza).
false.
```

4.

CODE:

```
com(X,Y):-
X>Y,write('X is Bigger Value');
X<Y,write('Y is Bigger Value').
```

OUTPUT:

```
?- com(8,9).
Y is Bigger Value
true.

?- com(80,9).
X is Bigger Value
true .

?- com(69,71).
Y is Bigger Value
true.

?- com(68,68).
```

```
false.
?- com(41,10).
X is Bigger Value
true
```

5. Prolog program using input/output function.

CODE:

```
position('jignesh','student').
position('silviya','teacher').
find_posiion:-write('Whose position you want to know?'),nl,
read(Input),nl,
position(Input,Output),nl,
write(Output).
```

OUTPUT:

```
?- position(X,Y).
X = jignesh,
Y = student .

?- position(jignesh,Y).
Y = student.

?- position(X,teacher).
X = silviya.

?- find_position.
Correct to: "find_posiion"? yes
Whose position you want to know?
|: jignesh.

student
true .
```

CONCLUSION:

Thus we have studied the prolog programming using mathematical operations and input/output functions on Prolog.

EXP NO. 09

AIM : Write a program to

perform Tokenization and Filtering in NLP.

THEORY:

1. Natural Language Processing (NLP)

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on enabling computers to understand, interpret, and generate human language in both written and spoken forms. It integrates concepts from linguistics, computer science, and machine learning to process natural language data effectively.

NLP aims to bridge the communication gap between humans and machines by converting unstructured language data into a structured format that machines can analyze. Human language is inherently complex due to ambiguity, context-dependence, and variations in syntax and semantics. NLP systems address these challenges through a sequence of processing stages.

The typical NLP pipeline includes:

- **Text preprocessing** (tokenization, filtering, normalization)
- **Syntactic analysis** (parsing, part-of-speech tagging)
- **Semantic analysis** (understanding meaning and context)
- **Pragmatic analysis** (interpreting intent based on context)

NLP techniques are widely applied in areas such as information retrieval, sentiment analysis, machine translation, speech recognition, and text summarization. The effectiveness of any NLP system largely depends on the quality of preprocessing steps, which prepare raw text for further analysis.

2. Tokenization

Tokenization is the process of breaking a stream of text into smaller meaningful units called tokens. These tokens serve as the basic building blocks for further NLP tasks. Tokenization can be performed at different levels depending on the application.

Types of Tokenization

1. **Sentence Tokenization**
   Divides text into sentences.
   Example:
   "NLP is useful. It is widely used." →
   ["NLP is useful.", "It is widely used."]
2. **Word Tokenization**
   Splits sentences into individual words.
   Example:
   "NLP is interesting" →
   ["NLP", "is", "interesting"]
3. **Character Tokenization**
   Breaks text into individual characters.
   Example:
   "NLP" → ["N", "L", "P"]
4. **Subword Tokenization**
   Divides words into smaller units (useful for handling unknown or rare words in modern NLP models).
   Example:
   "unhappiness" → ["un", "happi", "ness"]

Importance of Tokenization

- Converts raw text into a structured format
- Facilitates further processes such as parsing and feature extraction
- Helps in handling large text efficiently
- Essential for statistical and deep learning models

Challenges in Tokenization

- Handling punctuation and special symbols
- Dealing with contractions (e.g., "don't" → "do", "not")
- Language-specific variations
- Ambiguity in word boundaries (especially in languages without spaces)

Thus, tokenization is a critical preprocessing step that directly affects the performance of NLP systems.

3. Filtering

Filtering is the process of removing unwanted or less informative components from tokenized text to enhance data quality. It is also referred to as text cleaning or noise removal.

Components Removed During Filtering

1. **Stopwords Removal**
   Stopwords are commonly used words (e.g., *is, the, in, and*) that carry little semantic meaning. Removing them reduces data size and improves processing efficiency.
2. **Punctuation Removal**
   Symbols such as commas, periods, and exclamation marks are often removed unless they contribute to meaning (e.g., sentiment analysis).
3. **Special Character Removal**
   Characters like @, #, $, %, etc., are removed unless required (e.g., hashtags in social media analysis).
4. **Lowercasing**
   Converting all text to lowercase to maintain uniformity (e.g., "NLP" and "nlp" treated the same).
5. **Number Removal (optional)**
   Numbers may be removed depending on the application.

Example

Original sentence:
"This is an example of NLP processing!"

After tokenization:
["This", "is", "an", "example", "of", "NLP", "processing", "!"]

After filtering:
["example", "NLP", "processing"]

Importance of Filtering

- Reduces noise in the dataset
- Improves model accuracy and efficiency
- Decreases computational complexity
- Focuses on meaningful and relevant words

Limitations of Filtering

- Removing stopwords may sometimes lose important context
- Over-filtering can reduce information needed for certain tasks
- Some applications (e.g., sentiment analysis) may require punctuation or stopwords

Implementation in Python

Step 1: Import Required Libraries

```
from nltk.corpus import stopwords
import string

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

Step 2 : Define Text and Perform Tokenization

```
# Input text
a = input("Enter a text:")

# Tokenization
tokens = word_tokenize(a)
print("Tokens:", tokens)
```

OUTPUT :

```
Enter a text:Wow! Such a beautiful morning
Tokens: ['Wow', '!', 'Such', 'a', 'beautiful', 'morning']
```

Step 3 : Remove Stopwords and Punctuations

```
#Define stopwords and punctuations
stop_words = set(stopwords.words('english'))
punctuations = set(string.punctuation)

#Filter out stopwords and punctuations
filtered_tokens = []
for word in tokens:
    if word.lower() not in stop_words and word not in string.punctuation:
        filtered_tokens.append(word)

print("Filtered Tokens:", filtered_tokens)
```

OUTPUT :

```
Enter a text:Wow! Such a beautiful morning.
Tokens: ['Wow', '!', 'Such', 'a', 'beautiful', 'morning', '.']
Filtered Tokens: ['Wow', 'beautiful', 'morning']
```

CONCLUSION :

In this experiment, Natural Language Processing techniques were successfully implemented using the NLTK library in Python. The process of tokenization was performed to split the input text into smaller units called tokens, making the text easier to analyze and process. Additionally, filtering techniques such as removal of stopwords and punctuation were applied to eliminate unnecessary data and retain only meaningful words.